Java Collections Framework

Guia Completa con Codigo, Diagramas y Casos de Uso

Academia Backend & Data 2026

Autor: Miguel Rugerio

Generado con: Claude Code + Playwright

Fecha: 24 de febrero de 2026

Basado en el articulo de Shubham Jha (Medium)

Contenido

1. Introduccion al Java Collections Framework
2. Jerarquia de Interfaces y Clases
3. Interface List (ArrayList, LinkedList, Vector, Stack)
4. Interface Set (HashSet, LinkedHashSet, TreeSet)
5. Interface Queue
6. Interface Deque
7. Interface Map (HashMap, LinkedHashMap, TreeMap, ConcurrentHashMap)
8. Colecciones Concurrentes
9. Streams API con Colecciones
10. Tabla Comparativa de Complejidad (Big-O)
11. Cuando Usar Cada Coleccion

1. Introduccion al Java Collections Framework

El **Java Collections Framework (JCF)** es una arquitectura unificada para representar y manipular colecciones de objetos. Proporciona:

- **Interfaces core:** definen los tipos abstractos de colecciones (List, Set, Map, Queue)
- **Implementaciones concretas:** clases listas para usar (ArrayList, HashMap, TreeSet...)
- **Algoritmos:** metodos estaticos en la clase `Collections` (sort, shuffle, binarySearch...)

> **Paquete principal:** `java.util` — Todas las interfaces y clases del framework estan aqui. Para colecciones concurrentes: `java.util.concurrent`.

Antes de Java 1.2, solo existian `Vector`, `Hashtable` y arreglos. El JCF estandarizo el manejo de colecciones con una jerarquia coherente de interfaces.

2. Jerarquia de Interfaces y Clases

Este diagrama muestra la jerarquia completa del framework. Nota que **Map** es una interfaz separada que *no* extiende de Collection:

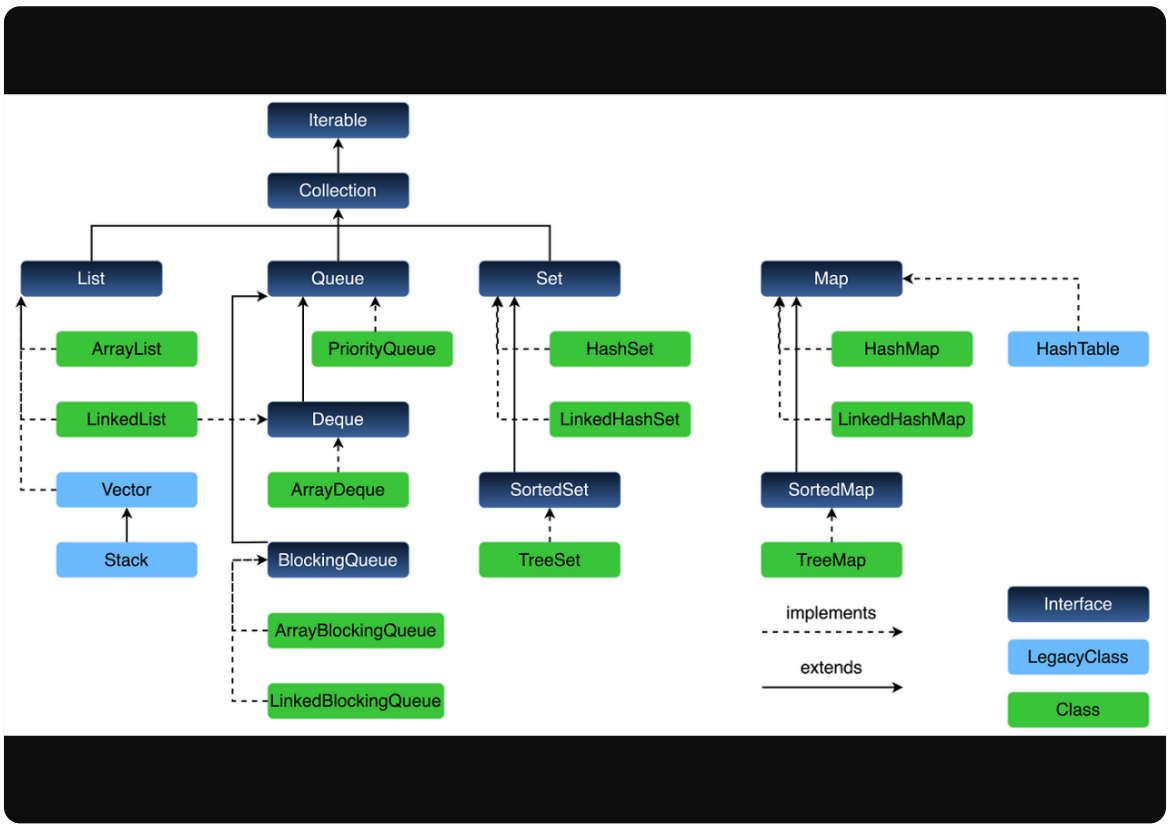


Diagrama: Jerarquia completa del Java Collections Framework

```
Iterable (interfaz raiz)
  └── Collection
      ├── List ─── ArrayList, LinkedList, Vector → Stack
      ├── Set ──── HashSet, LinkedHashSet, SortedSet → TreeSet
      └── Queue ── PriorityQueue, BlockingQueue
            └── Deque ── ArrayDeque, LinkedList

Map (interfaz independiente)
  ├── HashMap → LinkedHashMap
  ├── SortedMap → TreeMap
  ├── Hashtable (legacy)
  └── ConcurrentHashMap
```

> **Ojo:** Las clases en azul claro del diagrama (`Vector`, `Stack`, `Hashtable`) son **clases legacy**. Estan sincronizadas pero su uso se desaconseja en codigo moderno. Usa `ArrayList`, `ArrayDeque` y `HashMap` en su lugar.

3. Interface List

Una **List** es una coleccion *ordenada* que *permite elementos duplicados*. Cada elemento tiene un indice (posicion) empezando desde 0.

Caracteristicas principales

- Mantiene el **orden de insercion**
- **Permite duplicados**
- Acceso por **indice**: `get(i)`, `set(i, elem)`
- Implementaciones: `ArrayList`, `LinkedList`, `Vector`, `Stack`

ArrayList — Arreglo redimensionable

Aspecto	Detalle
Estructura interna	Arreglo (`Object[]`) que crece automaticamente
Acceso por indice	**O(1)** — acceso directo
Insercion al final	**O(1) amortizado** (O(n) cuando se redimensiona)
Insercion/eliminacion en medio	**O(n)** — requiere mover elementos
Thread-safe	No

Ejemplo de codigo — List con ArrayList

```java
import java.util.*;

public class ListExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Spring");
        list.add("Java"); // duplicates allowed
        System.out.println("List: " + list);
    }
}
```

Codigo: List permite duplicados ("Java" aparece dos veces)

Escrito como texto:

```java
import java.util.*;

public class ListExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Spring");
        list.add("Java");   // duplicados permitidos
        System.out.println("List: " + list);
        // Salida: List: [Java, Spring, Java]
    }
}
```

LinkedList — Lista doblemente enlazada

Aspecto	Detalle
Estructura interna	Nodos con punteros `prev` y `next`
Acceso por indice	**O(n)** — recorre nodos
Insercion/eliminacion al inicio/final	**O(1)**
Implementa tambien	`Deque` (doble extremo)

Cuando usar cual?
`ArrayList`: lecturas frecuentes con acceso aleatorio (get por indice).
`LinkedList`: inserciones/eliminaciones frecuentes al inicio o al final.

4. Interface Set

Un **Set** es una coleccion que *no permite elementos duplicados*. Utiliza `equals()` y `hashCode()` para determinar unicidad.

HashSet — Sin orden garantizado

Aspecto	Detalle
Estructura interna	Tabla hash (`HashMap` internamente)
Orden	**No garantizado**
add/remove/contains	**O(1)** promedio
Permite null	Si (un solo null)

```java
Set<String> hashSet = new HashSet<>();
hashSet.add("Java");
hashSet.add("Spring");
hashSet.add("Java");
System.out.println("HashSet: " + hashSet);
```

Codigo: HashSet ignora el duplicado "Java"

```java
Set<String> hashSet = new HashSet<>();
hashSet.add("Java");
hashSet.add("Spring");
hashSet.add("Java");     // duplicado: NO se agrega
System.out.println("HashSet: " + hashSet);
// Salida: HashSet: [Java, Spring] (orden no garantizado)
```

LinkedHashSet — Mantiene orden de insercion

```java
Set<String> linkedHashSet = new LinkedHashSet<>();
linkedHashSet.add("Java");
linkedHashSet.add("Spring");
System.out.println("LinkedHashSet: " + linkedHashSet);
```

Codigo: LinkedHashSet preserva el orden en que se insertaron los elementos

```java
Set<String> linkedHashSet = new LinkedHashSet<>();
linkedHashSet.add("Java");
linkedHashSet.add("Spring");
System.out.println("LinkedHashSet: " + linkedHashSet);
// Salida: LinkedHashSet: [Java, Spring] (orden de insercion)
```

TreeSet — Ordenado naturalmente

Aspecto	Detalle
Estructura interna	Arbol rojo-negro (Red-Black Tree)
Orden	**Orden natural** o por Comparator
add/remove/contains	**O(log n)**
Permite null	No (lanza NullPointerException)

```
Set<String> treeSet = new TreeSet<>();
treeSet.add("Banana");
treeSet.add("Apple");
treeSet.add("Mango");
System.out.println("TreeSet: " + treeSet); // Sorted output
```

Codigo: TreeSet ordena automaticamente los elementos

```
Set<String> treeSet = new TreeSet<>();
treeSet.add("Banana");
treeSet.add("Apple");
treeSet.add("Mango");
System.out.println("TreeSet: " + treeSet);
// Salida: TreeSet: [Apple, Banana, Mango] (sorted!)
```

Casos de uso de Set:

- Garantizar unicidad (IDs, correos, usernames)
- Almacenamiento de datos ordenados (TreeSet)
- Eliminar duplicados de una lista: `new HashSet<>(lista)`

5. Interface Queue

Una **Queue** (cola) sigue el principio **FIFO** (First-In, First-Out): el primer elemento que entra es el primero que sale.

Metodo	Descripcion	Si falla
`add(e)` / `offer(e)`	Agrega al final de la cola	Excepcion / false
`remove()` / `poll()`	Remueve y retorna el primero	Excepcion / null
`element()` / `peek()`	Consulta el primero sin remover	Excepcion / null

Implementaciones: LinkedList y PriorityQueue

```java
Queue<String> queue = new LinkedList<>();
queue.add("First");
queue.add("Second");
queue.add("Third");
System.out.println("Queue: " + queue);
System.out.println("Polled: " + queue.poll());
System.out.println("After Poll: " + queue);
```

Codigo: Queue con poll() remueve el primer elemento (FIFO)

```java
Queue<String> queue = new LinkedList<>();
queue.add("First");
queue.add("Second");
queue.add("Third");
System.out.println("Queue: " + queue);
System.out.println("Polled: " + queue.poll());
System.out.println("After Poll: " + queue);
// Queue: [First, Second, Third]
// Polled: First
// After Poll: [Second, Third]
```

> **Casos de uso:** Programacion de tareas (task scheduling), manejo de buffers, colas de mensajes, procesamiento por orden de llegada.

6. Interface Deque

Un **Deque** (Double-Ended Queue) permite insertar y remover elementos en **ambos extremos**. Puede usarse como Stack (LIFO) o como Queue (FIFO).

```java
Deque<String> deque = new ArrayDeque<>();
deque.addFirst("First");
deque.addLast("Last");
System.out.println("Deque: " + deque);
```

Codigo: Deque con addFirst() y addLast()

```java
Deque<String> deque = new ArrayDeque<>();
deque.addFirst("First");
deque.addLast("Last");
System.out.println("Deque: " + deque);
// Salida: Deque: [First, Last]
```

Metodo	Extremo	Descripcion
`addFirst(e)` / `offerFirst(e)`	Inicio	Agrega al inicio
`addLast(e)` / `offerLast(e)`	Final	Agrega al final
`removeFirst()` / `pollFirst()`	Inicio	Remueve del inicio
`removeLast()` / `pollLast()`	Final	Remueve del final

Recomendacion: Usa `ArrayDeque` en lugar de `Stack` (legacy). ArrayDeque es mas rapido y no tiene el overhead de sincronizacion de Stack/Vector.

7. Interface Map

Un **Map** almacena pares **clave-valor**. Las claves deben ser *unicas*; los valores pueden repetirse. **Map NO extiende Collection**.

HashMap — El mas usado

Aspecto	Detalle
Estructura interna	Tabla hash (buckets + listas/arboles)
Orden	**No garantizado**
put/get/remove	**O(1)** promedio
Permite null	Una clave null, multiples valores null
Thread-safe	No

```java
Map<Integer, String> hashMap = new HashMap<>();
hashMap.put(1, "Java");
hashMap.put(2, "Spring");
hashMap.put(1, "Hibernate");
System.out.println("HashMap: " + hashMap);
```

Codigo: HashMap sobrescribe el valor cuando se repite la clave (1 → "Hibernate" reemplaza "Java")

```
Map<Integer, String> hashMap = new HashMap<>();
hashMap.put(1, "Java");
hashMap.put(2, "Spring");
hashMap.put(1, "Hibernate");  // sobrescribe clave 1
System.out.println("HashMap: " + hashMap);
// Salida: HashMap: {1=Hibernate, 2=Spring}
```

LinkedHashMap — Orden de insercion

```
Map<Integer, String> linkedHashMap = new LinkedHashMap<>();
linkedHashMap.put(1, "Java");
linkedHashMap.put(2, "Spring");
System.out.println("LinkedHashMap: " + linkedHashMap);
```

Codigo: LinkedHashMap mantiene el orden en que se insertaron los pares

```
Map<Integer, String> linkedHashMap = new LinkedHashMap<>();
linkedHashMap.put(1, "Java");
linkedHashMap.put(2, "Spring");
System.out.println("LinkedHashMap: " + linkedHashMap);
// Salida: LinkedHashMap: {1=Java, 2=Spring} (orden de insercion)
```

TreeMap — Ordenado por clave

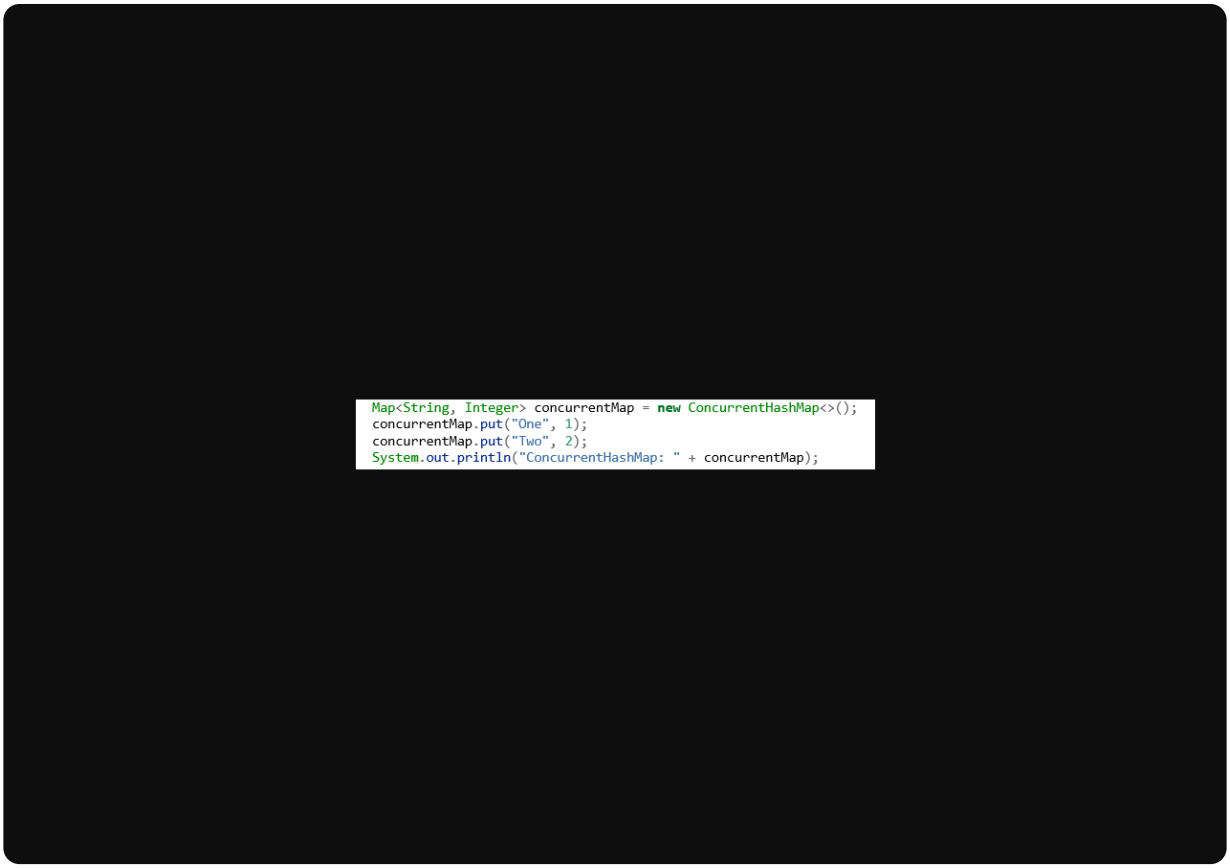
Aspecto	Detalle
Estructura interna	Arbol rojo-negro
Orden	**Orden natural de claves** (o Comparator)
put/get/remove	**O(log n)**
Permite null key	No

```
Map<String, Integer> treeMap = new TreeMap<>();
treeMap.put("Banana", 3);
treeMap.put("Apple", 2);
treeMap.put("Mango", 1);
System.out.println("TreeMap: " + treeMap);
```

Codigo: TreeMap ordena automaticamente por clave (Apple, Banana, Mango)

```
Map<String, Integer> treeMap = new TreeMap<>();
treeMap.put("Banana", 3);
treeMap.put("Apple", 2);
treeMap.put("Mango", 1);
System.out.println("TreeMap: " + treeMap);
// Salida: TreeMap: {Apple=2, Banana=3, Mango=1}
```

ConcurrentHashMap — Thread-safe



Codigo: ConcurrentHashMap para uso en aplicaciones multi-hilo

```
Map<String, Integer> concurrentMap = new ConcurrentHashMap<>();
concurrentMap.put("One", 1);
concurrentMap.put("Two", 2);
System.out.println("ConcurrentHashMap: " + concurrentMap);
// Thread-safe sin bloquear toda la tabla
```

Casos de uso de Map:

- Busqueda rapida por clave → `HashMap`
- Mantener orden de insercion → `LinkedHashMap`
- Mapas ordenados por clave → `TreeMap`
- Aplicaciones multi-hilo → `ConcurrentHashMap`

8. Colecciones Concurrentes

El paquete `java.util.concurrent` ofrece colecciones seguras para uso multi-hilo, sin el overhead de las colecciones legacy sincronizadas:

Coleccion	Equivalente no-concurrente	Estrategia
`ConcurrentHashMap`	HashMap	Segmentos (lock striping)
`CopyOnWriteArrayList`	ArrayList	Copia en cada escritura
`BlockingQueue`	Queue	Bloqueo en put/take
`ConcurrentLinkedQueue`	LinkedList	Lock-free (CAS)

Wrappers de la clase Collections

La clase utilitaria `Collections` ofrece metodos para crear versiones sincronizadas o inmutables:

```
// Sincronizar una lista existente
List<String> syncList = Collections.synchronizedList(new ArrayList<>());

// Crear un set inmutable (no modificable)
Set<String> readOnly = Collections.unmodifiableSet(mySet);

// Desde Java 9: factory methods
List<String> immutable = List.of("Java", "Spring");
Map<String, Integer> immutableMap = Map.of("Java", 1, "Spring", 2);
```

9. Streams API con Colecciones

Desde Java 8, el **Streams API** permite realizar operaciones de estilo funcional sobre colecciones:

```
List<String> names = Arrays.asList("Java", "Spring", "Hibernate");
names.stream().filter(n -> n.startsWith("J")).forEach(System.out::println);
```

Codigo: Streams con filter y forEach usando lambdas

```
List<String> names = Arrays.asList("Java", "Spring", "Hibernate");

// Filtrar nombres que empiezan con "J" e imprimir
names.stream()
     .filter(n -> n.startsWith("J"))
     .forEach(System.out::println);
// Salida: Java
```

Operaciones comunes con Streams:

Operacion	Tipo	Descripcion
`filter()`	Intermedia	Filtra elementos por condicion
`map()`	Intermedia	Transforma cada elemento
`sorted()`	Intermedia	Ordena los elementos
`distinct()`	Intermedia	Elimina duplicados
`forEach()`	Terminal	Ejecuta accion por cada elemento
`collect()`	Terminal	Recopila en otra coleccion
`reduce()`	Terminal	Reduce a un solo valor
`count()`	Terminal	Cuenta elementos

10. Tabla Comparativa de Complejidad (Big-O)

List

Operacion	ArrayList	LinkedList
get(index)	**O(1)**	O(n)
add(elem) al final	**O(1)***	**O(1)**
add(index, elem)	O(n)	**O(1)****
remove(index)	O(n)	**O(1)****
contains(elem)	O(n)	O(n)

* amortizado ** O(1) solo si ya tienes referencia al nodo; O(n) si buscas por indice

Set

Operacion	HashSet	LinkedHashSet	TreeSet
add(elem)	**O(1)**	**O(1)**	O(log n)
remove(elem)	**O(1)**	**O(1)**	O(log n)
contains(elem)	**O(1)**	**O(1)**	O(log n)
Orden	Ninguno	Insercion	Natural/Comparator

Map

Operacion	HashMap	LinkedHashMap	TreeMap	ConcurrentHashMap
put(k,v)	**O(1)**	**O(1)**	O(log n)	**O(1)**
get(key)	**O(1)**	**O(1)**	O(log n)	**O(1)**
remove(key)	**O(1)**	**O(1)**	O(log n)	**O(1)**
Orden	Ninguno	Insercion	Clave	Ninguno
Thread-safe	No	No	No	**Si**

11. Cuando Usar Cada Coleccion

Necesitas...	Usa	Razon
Lista ordenada con acceso rapido por indice	`ArrayList`	O(1) para get
Inserciones/eliminaciones frecuentes al inicio	`LinkedList`	O(1) para addFirst/ removeFirst
Conjunto sin duplicados	`HashSet`	O(1) para add/contains
Conjunto ordenado	`TreeSet`	Arbol rojo-negro, O(log n)
Conjunto con orden de insercion	`LinkedHashSet`	Hash + lista enlazada
Mapa clave-valor rapido	`HashMap`	O(1) promedio
Mapa ordenado por clave	`TreeMap`	Arbol rojo-negro
Mapa con orden de insercion	`LinkedHashMap`	Hash + lista enlazada
Mapa thread-safe	`ConcurrentHashMap`	Lock striping, alta concurrencia
Cola FIFO	`LinkedList` o `ArrayDeque`	FIFO simple
Cola con prioridad	`PriorityQueue`	Heap binario
Pila (Stack/LIFO)	`ArrayDeque`	Mas rapido que Stack
Lista thread-safe	`CopyOnWriteArrayList`	Lecturas frecuentes, pocas escrituras

Regla general:
- Por defecto usa `ArrayList`, `HashSet` y `HashMap`.
- Solo cambia a otra implementacion cuando necesites ordenamiento, concurrencia o patrones de acceso especificos.

Academia Backend & Data 2026

Miguel Rugerio — Generado con Claude Code + Playwright

24 de febrero de 2026

Basado en "Mastering Java Collections in Detail" por Shubham Jha